

LAB MANUAL

CSE 110: Object Oriented Programming

Course Instructor

Dr. Anup Kumar Paul,

Associate Professor

Dept. of Computer Science and Engineering



INTRODUCTION TO OOP

Object-oriented programming (OOP) is a computer science term used to characterize a programming language that began development in the 1960's. The term 'object-oriented programming' was originally coined by Xerox PARC to designate a computer application that describes the methodology of using objects as the foundation for computation. By the 1980s, OOP rose to prominence as the programming language of choice, exemplified by the success of C++. Currently, OOPs such as Java, J2EE, C++, C#, Visual Basic.NET, Python and Java Script are popular OOP programming languages that any career-oriented Software Engineer or developer should be familiar with. OOP is widely accepted as being far more flexible than other computer programming languages. OOPs use three basic concepts as the fundamentals for the Abstraction, Polymorphism, Event Handling and Encapsulation are also significant concepts within object-oriented programming languages that are explained in online tutorials describing the functionality of each concept in detail.

The java platform is undoubtedly fast moving and comprehensive. Its many application programming interfaces (APIs) provide a wealth of functionality for all aspects of application and system-level programming. Real-world developers never use one or two APIs to solve a problem but bring together key functionality spanning a number of APIs, knowing which APIs you need, which parts of which APIs you need, and how the APIs work together to create the best solution can be a daunting task.

GUIDELINES TO STUDENTS

1. Equipment in the lab for the use of the student community. Students need to maintain a proper decorum in the computer lab. Students must use the equipment with care. Any damage caused is punishable.
2. Students are instructed to come to the lab in formal dresses only.
3. Students are supposed to occupy the systems allotted to them and are not supposed to talk or make noise in the lab.
4. Students are required to carry their observation book and lab records with completed exercises while entering the lab.
5. Lab records need to be submitted every week.
6. Students are not supposed to use pen drives in the lab.

LAB OBJECTIVE

1. To introduce Java compiler and eclipse platform.
2. To make the student learn an object-oriented way of solving problems using java.
3. To make the students write programs using multithreading concepts and handle exceptions.
4. To make the students to create the Graphical User Interface using JAVA-FX

LAB OUTCOME

1. Able to use Java compiler and eclipse platform to write and execute java program.
2. Understand and Apply Object oriented features and Java concepts.
3. Able to apply the concept of multithreading and implement exception handling.
4. Develop applications using Console I/O and File I/O, GUI applications.

LIST OF EXPERIMENTS

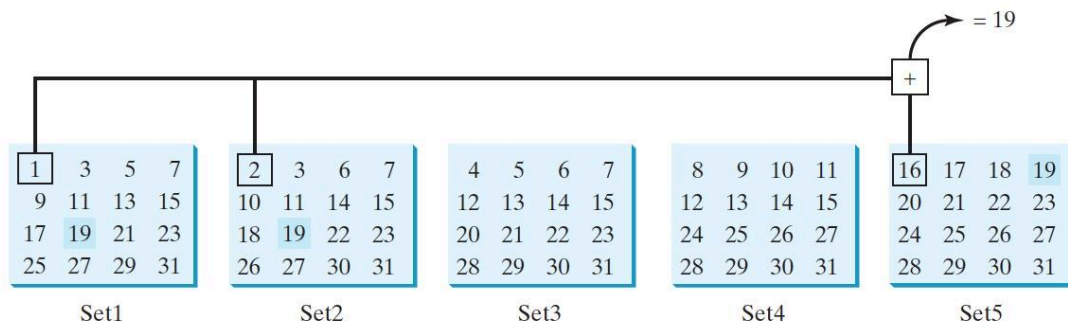
BASIC PROGRAM

1. a. Write a complete Java program that reads an array of integer numbers and finds the largest and smallest numbers from the array.
b. Write a complete Java program that reads a number and prints the reverse of the number. For example, if the input is 3245, the output will be 5423.
c. Write a complete Java program that print the pyramid of * given the input as a number of lines. For example if the input is 5, the output will be

```
      *
     ***
    *****
   ********
  **********
 **********
```

- d. Write a program which will guess the birthdate of a person. For example,

You can find out the date of the month when your friend was born by asking five questions. Each question asks whether the day is in one of the five sets of numbers.



The birthday is the sum of the first numbers in the sets where the day appears. For example, if the birthday is 19, it appears in Set1, Set2, and Set5. The first numbers in these three sets are 1, 2, and 16. Their sum is 19.

Objects and Classes

2. a. (The Rectangle class) Design a class named Rectangle to represent a rectangle. The class contains:

- ☐ Two double data fields named width and height that specify the width and height of the rectangle. The default values are 1 for both width and height.
- ☐ A no-arg constructor that creates a default rectangle.
- ☐ A constructor that creates a rectangle with the specified width and height.
- ☐ A method named getArea() that returns the area of this rectangle.
- ☐ A method named getPerimeter() that returns the perimeter.

Write a test program that creates two Rectangle objects—one with width 4 and height 40, and other with width 3.5 and height 35.9. Display the width, height, area, and perimeter of each rectangle in this order.

- b. (The Account class) Design a class named Account that contains:

- ☐ A private int data field named id for the account (default 0).
- ☐ A private double data field named balance for the account (default 0).
- ☐ A private double data field named annualInterestRate that stores the current interest rate (default 0). Assume that all accounts have the same interest rate.
- ☐ A private Date data field named dateCreated that stores the date when the account was created.
- ☐ A no-arg constructor creates a default account.
- ☐ A constructor that creates an account with the specified id and initial balance.
- ☐ The accessor and mutator methods for id, balance, and annualInterestRate.
- ☐ The accessor method for dateCreated.
- ☐ A method named getMonthlyInterestRate() returns the monthly interest rate.
- ☐ A method named getMonthlyInterest() returns the monthly interest.
- ☐ A method named withdraw that withdraws a specified amount from the account.
- ☐ A method named deposit that deposits a specified amount to the account.

Draw the UML diagram for the class then implement the class. (Hint: The method getMonthlyInterest() is to return monthly interest, not the interest rate. Monthly interest is $\text{balance} * \text{monthlyInterestRate}$. Monthly interest rate is $\text{annualInterestRate} / 12$. Note annualInterestRate is a percentage, for example 4.5%. You need to divide it by 100.)

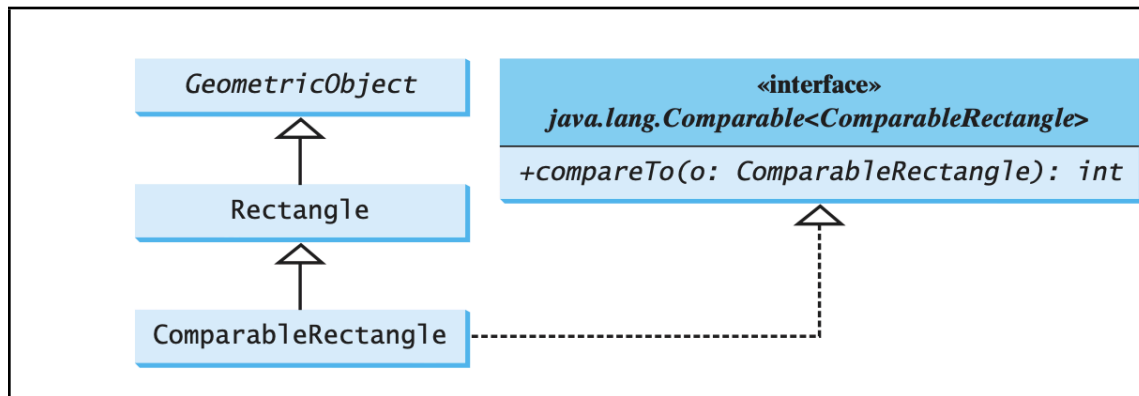
Write a test program that creates an Account object with an account ID of 1122, a balance of \$20,000, and an annual interest rate of 4.5%. Use the withdraw method to withdraw \$2,500, use the deposit method to deposit \$3,000, and print the balance, the monthly interest, and the date when this account was created.

INHERITANCE, POLYMORPHISM

3. a. Design a class to represent a Geometric object class as the superclass and design two other classes called Circle and Rectangle inherited from the superclass. Create objects of Circle and Rectangle classes and show the area and perimeter of Circle and Rectangle.
- b. Show the constructor chaining process by implementing the polymorphism feature of JAVA using the problem 7 class design.

ABSTRACT CLASS, INTERFACE

4. Write a java program to create an abstract class named GoemetricObject that contains three fields namely color, isFilled, and dateCreated and an abstract method named printArea(). Provide three classes named Rectangle, Triangle and Circle such that each one of the classes extends the class GoemetricObject. Each one of the classes contains their own properties and overrides the method printArea () that prints the area of the given object.
5. Write a java program to sort an array of objects by implementing Comparable<E> interface. Implement the following



Exception Handling and File Input / Output

6. a. Write a complete Java program that reads a bunch of numbers from a file (input.txt) and separates the even number and odd numbers and writes them into two separate files namely even.txt and odd.txt.
- b. Write a complete Java program to count the number of characters, number of words, and number of lines from a text file.

	Generics
7. a. A Generic Class for Adding Two Numbers Create a generic class called AddNumbers that can add two numbers of any data type. Create instances of the class and use them to add two integers and two doubles. Use wrapper classes (Number, Integer, Double etc.) to create instances. b. A Generic Method for Printing Arrays Create a generic method called printArray that can print any type of array. For example, create an array of integers and an array of strings, and use the method to print each array.	

	Inner Class and Lambda Expression
8. a. Implement the following functional interface by using Lambda Expression and without Lambda Expression. <pre>interface Drawable { public int draw(int n); }</pre> b. Given an ArrayList<E> type of objects, perform the following: i. Using Lambda Expression, print all the elements from the list. ii. Using Lambda Expression, show the sum of squares of elements from the list. iii. Using Lambda Expression, filter out odd and even numbers from the list. iv. Using Lambda Expression, find out the sum of squares of even and odd numbers.	

Multithreading

10. a. Write a java program that implements a multi-thread application that has three threads. First thread generates random integer 1000 times, second thread generates a character say “A” 1000 times and Third thread generate special symbol “*” 1000 times.

b. A shared resource may become corrupted if it is accessed simultaneously by multiple threads. The following example demonstrates the problem. Suppose you create and launch 100 threads, each of which adds a penny to an account. Define a class named **Account** to model the account, a class named **AddAPennyTask** to add a penny to the account, and a main class that creates and launches threads. The relationships of these classes are shown in Figure below.

Write a java program that solves the resource conflict problem using all three different options as below:

1. synchronized keyword
2. synchronized block and
3. lock mechanism..

